

Splines penalizados como puente entre estadística y Machine Learning: Splines y Boosting

GABRIEL FERNÁNDEZ LAGO, DANIEL LUGARESÍ PALOMARES, ANTÍA VEGA CREGO*

Máster en Técnicas Estadísticas

gabriel.fernandez.lago@rai.usc.es, daniel.lugaresi@rai.usc.es, antia.vega.crego@rai.usc.es

3 de mayo de 2026

Resumen

En este trabajo se estudia el método de *gradient boosting* por componentes en regresión con pérdida cuadrática, utilizando splines penalizados como base-learners. El boosting se interpreta como un procedimiento de optimización mediante descenso de gradiente en espacios funcionales, mientras que los P-splines permiten modelar relaciones no lineales presentes en los datos. Además, el enfoque incorpora selección automática de variables a través de su estructura iterativa. Se describe el desarrollo metodológico del modelo y se ilustra su comportamiento mediante una simulación en alta dimensión, analizando su capacidad para identificar variables relevantes y estimar efectos no lineales.

1. Introducción

El boosting se ha consolidado como uno de los algoritmos más potentes dentro del machine learning, al permitir construir modelos complejos mediante la combinación de modelos simples. Originalmente desarrollado para problemas de clasificación (Hastie et al. (2009)), su formulación moderna lo interpreta como un procedimiento para construir modelos de regresión aditivos (Bühlmann and Yu (2003)).

En este trabajo estudiamos el boosting por componentes en regresión, utilizando splines penalizados para capturar relaciones no lineales entre las variables (Schmid and Hothorn (2008)). Esta combinación permite construir modelos aditivos flexibles e interpretables, incorporando además selección automática de variables, resultando especialmente útil en contextos de alta dimensionalidad. Finalmente, ilustramos el método mediante una simulación con el paquete `mboost` de R (Hofner et al. (2014))

2. Desarrollo del método

En esta sección realizaremos una explicación metodológica del *gradient-boosting por componentes con pérdida cuadrática y base-learners basados en splines penalizados*. Este método puede interpretarse como un procedimiento de optimización en espacios funcionales, donde el predictor se construye iterativamente mediante la adición de funciones simples (base-learners). Nos centraremos en el caso de pérdida L_2 , en el que el gradiente se corresponde con los residuos, y utilizaremos como funciones base los splines penalizados.

2.1. Boosting y la función de pérdida

Consideremos un conjunto de observaciones $\{(x_i, y_i)\}_{i=1}^n \stackrel{iid}{\sim} (X, Y)$, donde cada observación y_i de la variable respuesta es unidimensional, y cada vector de covariables $x_i \in \mathbb{R}^p$. El objetivo es estimar una función de predicción

*Universidade de Santiago de Compostela

$F : \mathbb{R}^p \rightarrow \mathbb{R}$, de manera que se minimice el *coste esperado*

$$\mathbb{E}[\rho(Y, F(X))],$$

donde $\rho : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}^+$ es una *función de pérdida*, que asumiremos convexa y diferenciable respecto de $F(X)$.

El procedimiento de Boosting consiste (Bühlmann and Yu (2003)) en estimar dicha función $F(\cdot)$ minimizando el riesgo empírico

$$\frac{1}{n} \sum_{i=1}^n \rho(y_i, F(\mathbf{x}_i)),$$

sujeto a una restricción estructural sobre la clase de funciones consideradas. En particular, se impone que el estimador $\hat{F}$ admita una representación aditiva en términos de funciones simples (*base-learners*) de la forma

$$\hat{F}(\mathbf{x}) = \sum_{k=1}^M h_k(\mathbf{x}, \hat{\theta}_k).$$

Cada learner $h_k(\mathbf{x}, \hat{\theta}_k)$ es un estimador de la regresión que depende de un subconjunto de las covariables y tiene respuesta univariante.

Bajo la formulación anterior, la estimación de F se lleva a cabo mediante un procedimiento iterativo de descenso de gradiente funcional, en el que en cada etapa se incorpora un nuevo base-learner que aproxima la dirección del gradiente negativo del riesgo empírico.

2.2. Descenso de gradiente funcional y pérdida L_2 .

El método de descenso de gradiente funcional que se utiliza en el algoritmo de boosting se realiza de la siguiente forma: consideremos un espacio funcional $\mathcal{F}$ y base-learners $h_1, \dots, h_M$. Tomemos además una función de pérdida ρ , y fijemos una iteración inicial $\hat{F}^{[0]} \in \mathcal{F}$. En la iteración m se calcula el gradiente negativo del riesgo empírico, y se evalúa sobre el predictor actual $\hat{F}^{[m-1]}$, obteniendo el vector $U^{[m]} = (U_i^{[m]})_{i=1}^n$, con componentes

$$U_i^{[m]} = - \left. \frac{\partial \rho(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)} \right|_{F=\hat{F}^{[m-1]}}.$$

Ahora, dado que el vector $U^{[m]}$ representa la dirección de máximo descenso únicamente en las observaciones muestrales, ajustamos cada uno de los M base-learners a las observaciones $\{(U_i^{[m]}, \mathbf{x}_i)\}_{i=1}^n$, obteniendo M ajustes $\hat{h}_k^{[m]}$, $k = 1, \dots, M$.

Finalmente, escogemos el estimador $\hat{h}_{k^*}^{[m]}(\mathbf{x})$ que resulte en un mejor ajuste (por ejemplo, aquel cuyos valores ajustados $\hat{h}_{k^*}^{[m]}$ minimicen la RSS). Damos entonces un paso de longitud $0 < \nu \leq 1$ en la dirección estimada de máximo descenso, actualizando el modelo como

$$\hat{F}^{[m]}(\mathbf{x}) = \hat{F}^{[m-1]}(\mathbf{x}) + \nu \hat{h}_{k^*}^{[m]}(\mathbf{x}).$$

Este procedimiento se repite hasta un máximo de iteraciones m_{parada} .

Si consideramos la función de pérdida L_2 , o función de pérdida cuadrática,

$$\rho(Y, F(X)) = \frac{1}{2} (Y - F(X))^2,$$

entonces el gradiente negativo es simplemente el residuo

$$U_i^{[m]} = y_i - \hat{F}^{[m-1]}(\mathbf{x}_i),$$

lo cual proporciona mayor facilidad de computación que otras funciones de pérdida que se consideran en la literatura. En el resto de secciones estudiaremos el método de Boosting considerando pérdida cuadrática.

2.3. L_2 -Boosting por componentes y splines penalizados como base-learners

Cuando se pretende construir un predictor en un contexto de alta dimensión, el boosting por componentes es un procedimiento particularmente adecuado. Se trata de seleccionar base-learners que involucren, cada uno de ellos, una única variable explicativa, es decir

$$h_k(\mathbf{x}, \hat{\theta}_k) = h_k(x_k, \hat{\theta}_k), \quad k = 1, \dots, p.$$

Así, cuando el algoritmo selecciona el mejor predictor $\hat{h}_{k^*}^{[m]}(\mathbf{x})$, realiza una selección de

variables automática. Además, cuando tenemos relaciones no lineales, los splines de suavizado son base-learners especialmente apropiados al utilizar pérdida L_2 , como se demuestra en Bühlmann and Yu (2003). En particular, al considerar una cantidad pequeña de grados de libertad, df , podemos conseguir que sean learners débiles, es decir, que tengan sesgo alto y varianza baja, permitiendo que el algoritmo reduzca el sesgo iterativamente seleccionando en cada paso la covariable más relevante y haciendo el modelo progresivamente más flexible.

En Schmid and Hothorn (2008) se demuestra que emplear splines penalizados como base-learners produce una aproximación computacionalmente eficiente de los splines de suavizado en L_2 -Boosting, sin apenas perder capacidad predictiva. Así, considerando splines penalizados por componentes como base-learners, en la iteración m se ajusta cada P-Spline a las observaciones $\{(U_i^{[m]}, x_i)\}_{i=1}^n$, de forma que, para cada $k = 1, \dots, p$, se obtienen los valores ajustados

$$\hat{h}_k^{[m]} = B_k(B_k^\top B_k + \lambda K)^{-1} B_k^\top U^{[m]},$$

donde K es una matriz de penalización, λ es el parámetro de suavizado y B_k es la matriz de diseño, que proporciona el valor de las bases evaluadas en los puntos de la muestra. Se escoge entonces, para actualizar $\hat{F}^{[m]}$, el learner $\hat{h}_{k^*}^{[m]}(x)$ que minimice la RSS:

$$k^* = \operatorname{argmin}_k \left\| U^{[m]} - \hat{h}_k^{[m]} \right\|^2.$$

Con este procedimiento se obtiene un predictor final que será suma de splines que incluyen una única variable explicativa, es decir, un modelo con la misma forma estructural que un modelo GAM, lográndose una buena interpretabilidad y verificándose que la función predictora final mantendrá las propiedades de suavidad que el base-learner de partida.

2.4. Selección de parámetros del algoritmo

En cuanto a la selección de parámetros del algoritmo de boosting, cabe destacar el papel

clave del número máximo de iteraciones, m_{parada} , que controla la complejidad del modelo, pues es muy importante escoger adecuadamente su valor para evitar el sobreajuste de los datos. En la literatura se consideran dos metodologías para estimar este parámetro: técnicas basadas en el criterio AIC y técnicas de validación cruzada.

El número de iteraciones presenta una relación “compensatoria” con el parámetro de longitud de salto ν , en el sentido de que cuanto menor sea la longitud de salto, mayor será el número máximo de iteraciones necesario para obtener un buen ajuste. El valor de ν es de mucha menor importancia, basta con tomar un valor fijo pequeño (0,1 o 0,01), ya que valores demasiado grandes causan inestabilidad en el algoritmo, dando lugar a actualizaciones excesivas que sobrepasan la dirección de descenso óptima (*overshooting*).

3. Ilustración del método a través de simulaciones

Para realizar las simulaciones utilizaremos el paquete `mboost` de R (Hofner et al. (2014)). Generamos una muestra de tamaño $n = 500$ de una variable aleatoria $X \in \mathbb{R}^p$, con $p = 100$, con componentes i.i.d a una distribución $U(-1, 1)$. La variable respuesta se genera de acuerdo al siguiente modelo:

$$Y = \sin(X_1) + X_2^2 + X_3^3 + \varepsilon; \quad \varepsilon \sim N(0, 1).$$

En este modelo, únicamente tres de las cien variables tienen efecto real sobre la variable respuesta. El objetivo es comprobar si el algoritmo es capaz de seleccionar y recuperar el efecto de cada una de estas variables.

El modelo se ajusta mediante la función `gamboost()`, utilizando P-splines cúbicos como base-learners y con parámetro de longitud de salto $\nu = 0,1$. El número óptimo de iteraciones se ha seleccionado mediante validación cruzada, obteniéndose $m_{\text{parada}} = 73$.

La Tabla 1 muestra el número de veces que cada variable fue seleccionada a lo largo de las

73 iteraciones.

Variable	Selecciones
X1	34
X2	13
X3	14
X4	1
X9	3
X66	7
X75	1

Tabla 1: Frecuencia de selección de variables en el modelo de boosting.

El algoritmo selecciona en la mayoría de las iteraciones las variables X_1 , X_2 y X_3 , incorporando ruido de únicamente cuatro variables que no tienen efecto real. Así, los resultados ilustran la capacidad del método de seleccionar automáticamente las variables relevantes del modelo en un contexto de alta dimensión.

Como hemos explicado antes, este es un modelo con la misma forma estructural que un modelo GAM, por lo que podemos representar las contribuciones parciales de las variables X_1 , X_2 y X_3 estimadas por el modelo y las verdaderas contribuciones de los predictores a la variable respuesta, como se muestra en la Figura 1.

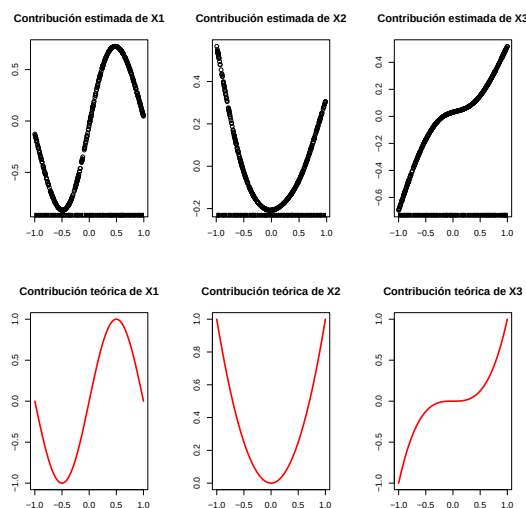


Figura 1: Contribuciones estimadas y reales de las variables X_1 , X_2 y X_3 .

Las funciones estimadas (en negro) reproducen adecuadamente las formas de las funciones reales (en rojo), es decir, la forma sinusoidal para la primera variable, la cuadrática para la segunda y la cúbica para la tercera, lo que ilustra que el método es capaz de capturar las relaciones no lineales reales entre los predictores y la respuesta.

Finalmente, el modelo obtiene un MSE de 0,953 y la varianza del error es $\sigma^2 = 1$. Esto indica que el modelo ha capturado prácticamente toda la señal real (el error residual podemos asumir que proviene del error normal de cada observación).

Referencias

- Bühlmann, P. and Yu, B. (2003). Boosting with the L2 loss: Regression and classification. *Journal of the American Statistical Association*, 98(462):324–339.
- Hastie, T., Tibshirani, R., and Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2nd edition.
- Hofner, B., Mayr, A., Robinzonov, N., and Schmid, M. (2014). Model-based boosting in R: A hands-on tutorial using the R package mboost. *Computational Statistics*, 29(1-2):3–35.
- Schmid, M. and Hothorn, T. (2008). Boosting additive models using component-wise P-splines. *Computational Statistics & Data Analysis*, 53(2):298–311.